

Homework #4: Deep Learning with 3D Point-clouds [100 points]
Due Date: Wednesday, 3 June 2026 (11:59 PM PT)

No late days are available for this homework.

Problem 1. Deep Learning with Point-clouds [100 points]

The purpose of this problem set is to explore modern deep-learning techniques that can be applied to 3D point-cloud data. To this end, we start our exploration with PointNet [1], a neural network architecture that applies deep learning on point sets, by encoding point (and other) sets in a permutation-invariant manner. By utilizing the basic ideas presented in PointNet we will first construct deep Autoencoders (AEs) operating on point-clouds. AEs are an elegant solution to the problem of data-compression – features extracted from them have found use in several practical applications [2]. The point-clouds of interest in this homework will be sampled from the surface of man-made objects: such as chairs, lamps, etc. that have well-defined semantic parts like the legs, or arm-rests of a chair. To exploit this compositionality of 3D articulated objects, we will further experiment with an AE that has the capacity to ‘segment’ point clouds into their constituent parts while also reconstructing the complete input object. Overall, you will evaluate the effect of different design choices for the learned representations and last, in an open-ended bonus question, you will be challenged to modify the given designs to come up with a more ‘part-aware’ embedding.

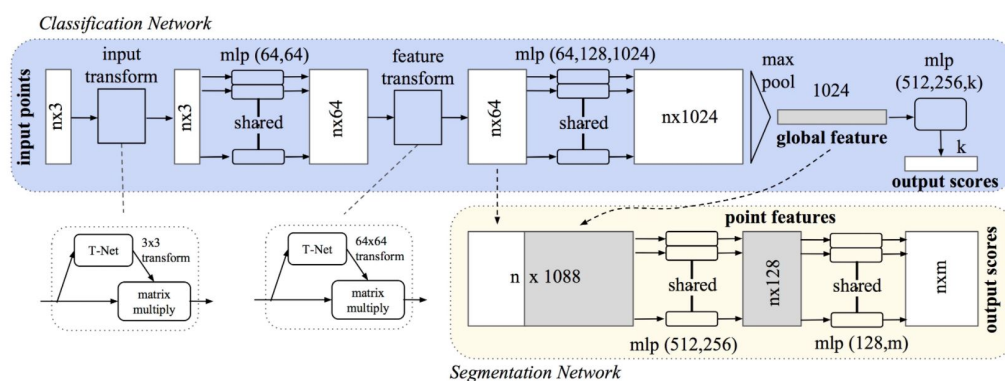


Figure 1: Network schematic of the PointNet’s *original* architecture (Figure 2, in [1]) showing the networks built to perform classification and segmentation of 3D point-clouds. For the purposes of this homework we will not use any input or feature transform (T_nets). Also, we will use a custom set of hyper-parameters that better fit the needs of our dataset.

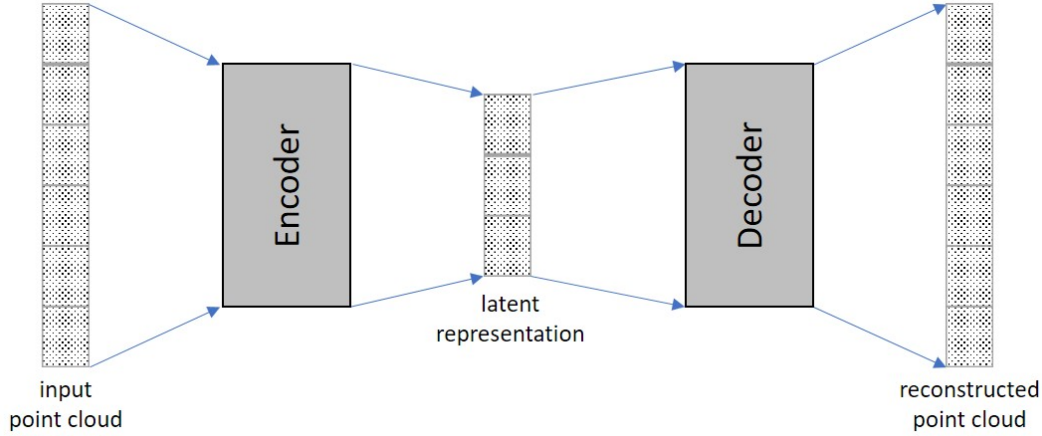


Figure 2: Network schematic of the encoder-decoder structure for a point-cloud AE.

Autoencoders. A deep AutoEncoder (AE) (Figure 2) is a deep neural network that tries to first compress and then reconstruct its input. In doing so, it first embeds the input into a space with significantly smaller dimensionality. This ‘squeezing’ forces the network to learn important features (abundant in the data) and happens at the end of the so called **encoder**, part of the AE. The other part of the AE, the **decoder**, is responsible for reconstructing the original input from the squeezed signal. To evaluate the efficacy of reconstruction of an AE, we need a distance function between (in our case) point-sets, which we will apply between the reconstructed and the given input signal [3]. The symmetric Chamfer pseudo-Distance (CD) presents a reliable and a computationally friendly choice to this end. Given two 3D point-clouds **A** and **B**, the CD is defined as:

$$d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B}) = \left(\sum_{x_B \in \mathbf{B}} \min_{x_A \in \mathbf{A}} |x_B - x_A|^2 \right) + \left(\sum_{x_A \in \mathbf{A}} \min_{x_B \in \mathbf{B}} |x_B - x_A|^2 \right). \quad (1)$$

In words, CD assigns each point of A to its nearest neighbor point in B (and vice versa) and sums the corresponding squared distances. Technically, CD is not a distance metric since it does not satisfy the triangle inequality. However, it is fast to compute and yields good results in practice.

Autoencoders & Segmentation. Man-made 3D objects are a result of human creativity satisfying both functional and aesthetic desiderata that are further constrained by physical and economical constraints. One result of these constraints is the compositionality of the 3D objects into constituent semantic parts (leg pieces, back types, etc.). ShapeNet [4] has an extensive annotation of such semantic parts for a variety of object categories [5], that we will exploit in this assignment. Concretely, we want to augment the learning of an AE operating on entire shapes, by training it to also predict the part type that each input point belongs to. Given adequate supervision, one way to achieve this is to “split” the decoder part of the AE, into two branches: one responsible for doing (entire) shape reconstruction and one for doing part-prediction of

points as shown in Figure 3. While the former decoding branch can be guided by the CD (as a typical decoder of an AE), the new, latter branch, can be optimized via the widely used classification loss of cross-entropy. As a result, the encoder will be optimized by both criteria simultaneously. The definition of cross-entropy follows. For two probability distributions p, q with support on a discrete space X ($\sum_{x \in X} p(x) = \sum_{x \in X} q(x) = 1$ and $p(x), q(x) > 0, \forall x \in X$), the cross entropy $H(p, q)$, is given by:

$$H(p, q) = - \sum_{x \in X} p(x) \log(q(x)). \quad (2)$$

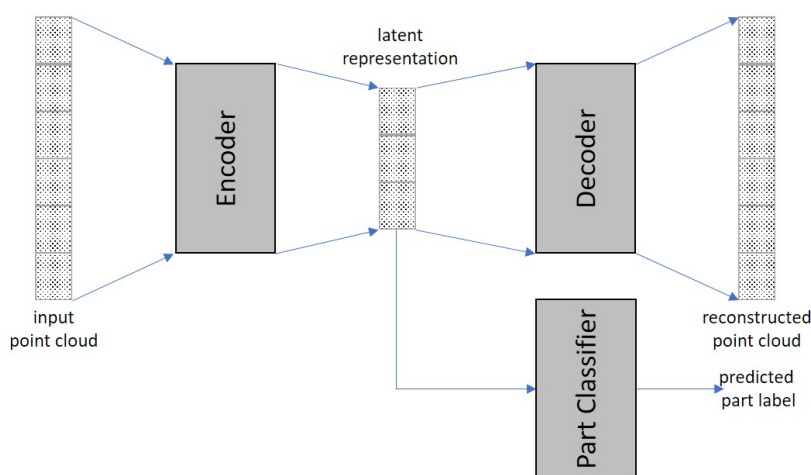


Figure 3: Network schematic of an AE with two branches: one for reconstruction of point-clouds and another for part prediction.

Dataset and Code. You will be given a dataset of one thousand point-clouds extracted from chair CAD models from the ShapeNet repository. The dataset has been split into three disjoint subsets that you will use for training, validation and testing purposes. Each point-cloud is comprised of 1024 points which are additionally labeled according to the semantic part they belong to (e.g. 1 if a point belongs to the "legs" part). Furthermore you are provided with a "golden" distance function $d_{\mathcal{P}}$ between every semantic part of every given point-cloud, that you will use to evaluate the part-awareness of your learned representations. The provided starter code is written and tested under Python 3.6 (any Python 3.x *should* work) and has one key dependency: PyTorch (any version above 1.3+ *should* work). If you haven't worked with PyTorch before, it may be helpful to read through a tutorial on getting started¹. Also, please read the *Coding Remarks* at the end of this assignment before beginning on the questions below.

The learning part (using PyTorch) is somewhat computationally heavy and as such it will be good to utilize GPUs to solve it efficiently. For this, credits for the Google Cloud Platform

¹https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

(GCP) will be given to each student, please find the detailed tutorial in ‘google cloud tutorial.pptx’. We recommend debugging/evaluating your pipelines locally (most likely on a CPU) and primarily use GCP for training purposes.

- a) (10 points) Show that the Chamfer distance $d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B})$ is not a true distance metric. Also derive the gradient $\nabla_{\mathbf{A}} d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B})$ in closed form.
- b) (5 points) Describe why the “global” feature in Figure 1 is invariant to the order of the points of the input point-cloud and mention two more functions (other than max-pool) that can preserve this invariance. If the max-pool is applied on features of dimension k , how many of the input points can affect the final values of the extracted feature?
- c) (20 points) An implementation of the Chamfer distance has been provided for you. Use `nn.torch.Linear` and `nn.torch.Conv1d` as the basic building blocks to implement the decoder and the encoder part of your AE. Make a decoder that is a multi-layer perceptron² with two *hidden* layers of 256 and 384 neurons each, each followed by the ReLU non-linearity. Implement a PointNet-like encoder with 5 layers of independent (per point) convolutions of dimensions [32, 64, 64, 128, 128] (use ReLU after each convolution) followed by a max-pool. To get all 20 points (and be able to solve any of the remaining questions), also prepare the code to train/save/evaluate the AE.
- d) (10 points) Train your AE network for 400 epochs and plot the reconstruction loss for the train-test-val splits at the end of every epoch. Which epoch achieved the best validation loss? Did the training loss improve after that epoch? How many epochs of training could you avoid (thus reducing the compute time) without significantly hurting the test reconstruction? In a few sentences, describe the trends observed by your plots.
- e) (15 points) Using the optimal model (according to validation loss), visualize the input point-clouds along with the resulting reconstructions for the 5 indicated (see starter code) point-clouds of the test-set. Do the reconstructions suffer from systemic errors? Further, visualize the worst and best reconstructed point-clouds (according to CD) along with their achieved Chamfer distances. Why is the worst reconstruction (so much) worse than the best one? Next, extract the latent vectors at the end of the encoder for the point-clouds in the test-split and compute their 2D T-SNE embeddings. How do the T-SNE neighborhoods look? You can use sklearn’s corresponding function³ to make the T-SNE plot and utilize the provided rendered images of each model.
- f) (20 points) Adapt the AE to also perform point-cloud part-segmentation. To this end, reconstruct the input point-cloud as in a typical AE, but also learn to output a second signal: a part label for **each** input point. Use your established AE architecture for reconstruction but now further concatenate the encoder’s output (“latent vector”) with each

²https://en.wikipedia.org/wiki/Multilayer_perceptron

³<http://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

input point and forward this combined signal to a new decoding branch that is trained to do the segmentation using the cross-entropy loss. Let the new branch be comprised by a single (hidden) convolutional layer that is applied independently to each point/latent vector combination and which results in a 64 dimensional (hidden) feature. Train the system for 400 epochs and plot the learning curves for each of the two losses independently, similar to how was done in part (d). Use the optimal model (per the validation set) according to the joint loss (summing the reconstruction one with the cross-entropy): Did the average reconstruction quality improve in comparison to (d)? Why? What is the average, per-point, segmentation accuracy on the test-set? Of the two learning tasks, which one seems to learn/generalize better? Make a plot depicting in color the part-predictions of the AE for the same (test) point-clouds of part (e) and provide brief comments on your findings.

- g) (20 points) Using the provided part-distance function $d_{\mathcal{P}}$, compare the cumulative distances of the encoding space learned by the vanilla AE of (d) vs. the part-aware AE of (f). Compute the cumulative distance of an encoding space by accumulating the part distances of the parts of every chair in the test split, to those of its nearest neighbor (NN) in the encoding. Use the Euclidean distance between the latent vectors to compute the neighborhoods. Let $M(A)$ denote all parts of chair A and $\tilde{M}(A, k)$ its k -th part. Define the one-way (part-based) distance of chair A from B as:

$$\sum_{k \in M(A) \cap M(B)} d_{\mathcal{P}}(\tilde{M}(A, k), \tilde{M}(B, k)) + \max_u \sum_{k \in M(A) \setminus M(B)} d_{\mathcal{P}}(\tilde{M}(A, k), \tilde{M}(B, u)).$$

Furthermore, compute the average number of semantic parts types that are shared between each chair and its neighbor ($|M(A) \cap M(B)|$), i.e., count how many times the matched point-clouds both have arm-rests, backs, etc. Lastly, report the average (latent) Euclidean distance between each chair to its matched neighbor. Compare your findings on the embeddings stemming from (d) and (f).

Coding Remarks. Please carefully follow the README to set things up properly. We highly recommend that you install the code as a module **inside** a virtual environment⁴. The starting place for your code should be the directory of notebooks, specifically the *main.ipynb* (or the *main.py* under *notebooks_as_python_scripts* if you prefer to work without a notebook). For parts (e) and (g) see *tsne_plot_with_latent_codes* and *measuring_part_awareness*. Pay extra attention in the comments starting with “students”.

⁴<https://docs.python.org/3/library/venv.html>

References

- [1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 652–660, 2017.
- [2] P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. Guibas, “Learning representations and generative models for 3d point clouds,” in *International conference on machine learning*, pp. 40–49, PMLR, 2018.
- [3] H. Fan, H. Su, and L. J. Guibas, “A point set generation network for 3d object reconstruction from a single image,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 605–613, 2017.
- [4] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, *et al.*, “Shapenet: An information-rich 3d model repository,” *arXiv preprint arXiv:1512.03012*, 2015.
- [5] L. Yi, V. G. Kim, D. Ceylan, I.-C. Shen, M. Yan, H. Su, C. Lu, Q. Huang, A. Sheffer, and L. Guibas, “A scalable active framework for region annotation in 3d shape collections,” *ACM Transactions on Graphics (ToG)*, vol. 35, no. 6, pp. 1–12, 2016.